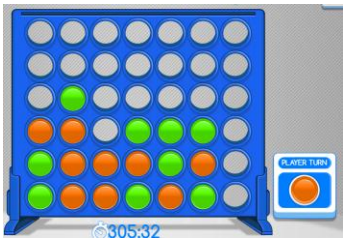
CS-3083 Project 2 (300 pts)

In this project, you will use Python to implement game tree search algorithms and algorithms for constraint satisfaction problems

You can only use the following Python modules in your programs:

- numpy
- matplotlib.pyplot
- time
- os
- json
- Tkinter or Pygame to visualize the game in Task1 (Optional)

Task 1: Game of Connect 4 (150 pts)



In this task you need to implement game tree search algorithms to design a computer player (agent) that can play with a human opponent. You need to implement both Alpha-Beta pruning search (AB) and Monte Carlo Tree Search (MCTS).

To familiarize yourself with this game, please open the website https://www.calculators.org/games/connect-4/ , and play a few rounds with the computer.

This game has a maximum branch factor of 7 and a maximum depth of 42 plys.

For the AB program, you will need to determine if a search depth cutoff should be applied such that the agent response is not too slow. (Do NOT let the human player wait for too long!) If a cutoff is indeed required, you will have to design a proper evaluation function. Obviously, the connect 4 does not have any pieces caught by opponents and removed during the game, so, the counting of pieces or material values working well for the game of chess will not work here. Instead, the patterns of piece distribution from both sides might be something to analyze by your evaluation function.

For the MCTS program, you will need to adopt a proper playout policy for the simulation step (Note: the 4 steps are selection, expansion, simulation, and back-propagation). The following are the two options of playout you must implement and test in your program:

- Option1: both sides randomly add pieces to available columns, i.e. no strategy at all;

- Option2: if the current player has a move that can defeat the opponent, then take it. Else, if the opponent has a move that can defeat the current player, the current player should try to block it. Otherwise, the current player takes a random move.

You are encouraged to design and adopt other playout policies, which should target improving the quality of simulation.

For both programs, there are control parameters that you must tune to achieve the best performance. For example, the AB program may need to set a proper cutoff depth, and the MCTS program has a constant C in its UCB1 computing. A good computer player (agent) should be responsive and reasonably smart. Record how you tuned these parameters and relevant experimental data to include in your report.

As a minimum requirement, your program must have a command-line style interface, which can show the progress of the game, allow human player to conveniently enter their moves. Upon reaching a terminal position, your program must immediately terminate the game and declare a winner or a draw.

Optionally, you may design a playable window easy for human players to participate.

A few hints:

- The board can be defined as a 6-by-7 numpy 2D array
- Make a minimum version work first, then add components and functions to improve it

Submission Requirements

1. Your python program (name it connect4_YourLastName_YourFirstName.py)
2. A report that covers the following contents:

(1) AB

- Overall design of the program
- How you designed the evaluation function. Elaborate how the evaluation function connects with the piece distribution of each side, and why the calculated scores reflect the chances of win or lose.
- How you balanced between the responsiveness and the smartness of agent. Provide experimental data to support your decision
- Play with your agent with your best ability. Provide a screenshot of computer winning and a screenshot of computer losing

(2) MCTS

- Overall design of the program
- How you designed and implemented the playout policies. How they performed differently in your test
- How you tuned any tunable control parameters to improve your program. Provide plots and texts to illustrate your findings

- Choose a moment near the end of one playout, provide the win#/playout# for each node in the game tree at that moment. You should store nodes and their win#/playout# clearly in a text file and upload it along with other files.
- Play with your agent with your best ability. Provide a screenshot of computer winning and a screenshot of computer losing

Task 2: Sudoku Solver (150 pts)

In this task, you will implement the backtracking search algorithm to solve Sudoku puzzles. The input puzzles will be generated by a SudokuGenerator.py (provided for the project). Upon finishing, print both the original puzzle and the solved pattern on screen for verification.
As you have learned from the lecture, the backtracking search can be accelerated with different strategies. In this task, while maintaining correctness, you must implement each of them to make your solver faster.

- SELECT-UNSIGNED-VARIABLE: you must compare random selection with the minimum remaining value (MRV) heuristic
- INFERENCE: you must compare forward-checking-only with the maintaining-arc-consistency (MAC) heuristic
- ORDER-DOMAIN-VALUES: you must compare the standard order with the least-constraining-value heuristic

SudokuGenerator.py

This is the provided python program to randomly generate Sudoku puzzles. While you know the 9-by-9 puzzles (block 3-by-3) from the textbook, this generator gives you more options by allowing you to choose the block size and set the number of missing values (empty squares)
The format to use the generator:

```
python SudokuGenerator.py 20 --output_file sudoku9.json --block_size 3
```

, where 20 is the number of empty squares, sudoku9.json is the file name, and block size is set 3, which defines a 9-by-9 puzzle with available numbers 1~9 for filling. Please use the following script to load the json file, which will be stored as a python nested list.

```
import json
with open('sudoku9.json', 'r') as f:
    board = json.load(f) # board will be a nested python list
```

One possible puzzle generated this way and its solution:

```
----------original question---------
 0   0   5   0   1   4   7   3   6
 4   7   0   6   8   3   2   1   0
 6   1   0   5   0   2   4   9   0
 1   6   4   0   3   0   5   7   2
 0   9   8   0   6   5   3   4   0
 3   5   2   0   0   7   6   8   9
 5   4   6   7   9   1   8   0   3
 9   0   7   0   5   8   1   6   0
 8   3   1   0   2   6   9   5   7
```

```
----------solution------------
 2   8   5   9   1   4   7   3   6
 4   7   9   6   8   3   2   1   5
 6   1   3   5   7   2   4   9   8
 1   6   4   8   3   9   5   7   2
 7   9   8   2   6   5   3   4   1
 3   5   2   1   4   7   6   8   9
 5   4   6   7   9   1   8   2   3
 9   2   7   3   5   8   1   6   4
 8   3   1   4   2   6   9   5   7
```

(Note: the zeros in the original list are empty squares to be filled by your program)

You can change the block_size to 4 or 5 for larger puzzles. With block_size = 4, the available numbers for filling are 1~16, and the size of the puzzle is 16-by-16. In the solution, you will need to make each block/row/column containing all the 16 values. Block_size = 5 can be interpreted similarly.

Evaluation Requirements:

You must compare each optimization with the non-optimized version. For each block size, tune the number of empty squares to thoroughly test the scalability of your program.

Submission Requirements:

1. Your python program (please name it "Sudoku_Solver_YourLastName_YourFirstName.py")
2. A report summarizing your implementation and findings:

(1) How you implemented the program

(2) What data structures you used to guarantee efficient storage/retrieval of data

(3) Elaborate the mechanism you adopted to apply and remove inference results

(4) Thorough performance analysis for different optimization settings. You should include both plots and texts to help illustrate and explain your findings.

(5) Experimental data, plots, and analysis for your program's scalability. How it performs under different settings of block size and number of missing empty squares. Note that the 9-by-9 puzzle can be set with at most 81 empty squares, and it is 256 for 16-by-16 puzzles, and 625 for 25-by-25 puzzles.

3. A summary about what you learned from the entire project 2.